

File Maintenance Tool

A Windows-first, unattended file maintenance utility for scheduled cleanup jobs. The core cleanup logic is isolated from OS-specific setup, notification, path, and disk-space behavior through the `internal/platform` abstraction.

The tool can:

- Scan configured file and folder paths.
- Identify files older than the configured retention period.
- Back up eligible files before deletion when backup is enabled for that path.
- Delete eligible files directly when backup is intentionally disabled for that path.
- Write operational, error, and count logs with retention cleanup.
- Launch a Windows setup wizard on first run when `config.ini` is missing.
- Print build metadata with the `-version` flag.

Current Status

This project is currently optimized for Windows scheduled execution.

Area	Current status
Windows setup wizard	Implemented with embedded PowerShell and Windows Forms.
Windows notifications	Implemented through PowerShell / Windows Forms message boxes.
Windows backup space validation	Implemented through <code>GetDiskFreeSpaceEx</code> .
Linux/macOS config check	Implemented as a safe <code>config.ini</code> existence check.
Linux/macOS backup space validation	Not implemented yet. Backup-enabled runs on those platforms will fail once space validation is reached.
GitHub Actions build matrix	Windows amd64, Linux amd64, macOS amd64, and macOS arm64.

Current Default Layout

The application currently uses portable defaults beside the executable:

```
fileMaintenance.exe
config/
  config.ini
  logging.json      # optional
logs/
  maintenance_YYYY-MM-DD.log
```

```
errors_YYYY-MM-DD.log  
count_YYYY-MM-DD.log
```

This is intentional. The platform package still supports OS-conventional directories such as AppData on Windows, but `cmd/main/main.go` currently defaults to `<exe>/config` and `<exe>/logs` to make Task Scheduler deployments and portable runs easier to reason about.

Override paths with:

```
fileMaintenance.exe -config-dir "C:\path\to\config" -log-dir "C:\path\to\logs"
```

First-Time Setup

On Windows, startup checks whether this file exists:

```
<config-dir>/config.ini
```

If it is missing, the Windows platform implementation launches the embedded PowerShell setup wizard. The wizard creates `config.ini`, and the application continues only if the file was created successfully.

On Linux and macOS, no GUI setup wizard is currently provided. If `config.ini` is missing, the platform implementation returns `false` and the application exits safely without processing files.

How It Works

1. Startup

- Resolve the active OS platform implementation.
- Resolve the executable directory.
- Set portable default paths for `config/` and `logs/`.
- Parse CLI flags.
- Initialize logging.

2. Configuration Check

- Call `platform.EnsureConfig(configDir, exeDir)`.
- On Windows, launch the setup wizard if `config.ini` is missing.
- On Linux/macOS, exit safely if `config.ini` is missing.

3. Configuration Loading

- Read `config.ini`.
- Parse `[backup]`, `[paths]`, and optional `[settings]` / `[advanced]` sections.
- Apply non-zero config values over CLI/default values.

4. Safety Checks

- Determine whether any configured path has backup enabled.
- If backup is required, validate the backup destination before deleting anything.
- Show a platform-specific critical notification if the backup path is inaccessible.

5. Maintenance Worker

- Scan configured paths using bounded walkers.
- Process file operations through one serialized processor.
- Copy before delete when backup is enabled.
- Delete directly only when backup is disabled for that path.
- Track per-path delete counts.

6. Cleanup and Exit

- Prune old logs according to log retention.
- Exit with an error code when fatal configuration or worker errors occur.

Execution Flow

The high-level Mermaid source is maintained in:

```
docs/diagrams/execution-flow-high-level.md
```

The detailed execution flow is maintained in:

```
docs/diagrams/execution-flow.md
```

The execution flow uses concurrent path discovery but serialized file operations. This avoids uncontrolled SMB/network load and keeps backup/delete behavior predictable.

Command-Line Flags

Retention and Logging

Flag	Default	Description
<code>-days</code>	<code>7</code>	Only files older than this many days are eligible. <code>0</code> effectively selects files older than the current instant.
<code>-log-retention</code>	<code>30</code>	Number of days to keep log files.
<code>-no-logs</code>	<code>false</code>	Disable file logging and write to stdout/stderr.

Flag	Default	Description
<code>-version</code>	<code>false</code>	Print version metadata and exit.

Paths

Flag	Default	Description
<code>-config-dir</code>	<code><exe>/config</code>	Directory containing <code>config.ini</code> and optional <code>logging.json</code> .
<code>-log-dir</code>	<code><exe>/logs</code>	Directory where log files are written.

Resource Controls

Flag	Default	Description
<code>-walkers</code>	<code>1</code>	Number of concurrent path walkers.
<code>-queue-size</code>	<code>300</code>	Buffered job queue size.
<code>-max-files</code>	<code>0</code>	Maximum jobs to handle in one run. <code>0</code> means unlimited.
<code>-max-runtime</code>	<code>30m</code>	Maximum run duration. <code>0</code> means unlimited. CLI values use Go duration strings such as <code>55m</code> , <code>1h</code> , or <code>30s</code> .
<code>-cooldown</code>	<code>0</code>	Delay after each processed job. Useful for SMB/network pacing. CLI values use Go duration strings such as <code>50ms</code> or <code>1s</code> .
<code>-retries</code>	<code>2</code>	Number of backup copy retries.

Important: after CLI flags are parsed, non-zero values from `config.ini` override CLI/default values. Confirm `[settings]` and `[advanced]` before relying on Task Scheduler arguments.

Configuration

`config/config.ini`

Required sections:

```
[backup]
path=D:\backups

[paths]
C:\Temp\OldFiles, yes
C:\Temp\ToDelete, no
```

Optional sections:

```
[settings]
days=7
log-retention=30

[advanced]
walkers=1
queue-size=300
retries=2
cooldown=50
max-files=0
max-runtime=3300000
```

Current implementation note: `[advanced]` duration values are parsed as milliseconds in `config.ini`. For example:

Setting	Meaning
<code>cooldown=50</code>	50 milliseconds.
<code>max-runtime=3300000</code>	55 minutes.
<code>max-runtime=0</code>	Do not override the CLI/default runtime limit.

CLI flags use duration strings like `-max-runtime 55m` and `-cooldown 50ms`; `config.ini` currently uses numeric milliseconds.

[backup]

Key	Description
<code>path</code>	Backup destination root path. Can be local or network/SMB.

[paths]

Each standalone line is a file or folder path followed by optional backup behavior:

```
path, yes|no
```

Value	Meaning
<code>yes</code>	Back up eligible files before deletion.
<code>no</code>	Delete eligible files without backup. Use carefully.
omitted	Backup defaults to enabled.

Supported path types:

Type	Example	Behavior
Folder	C:\Temp\OldFiles, yes	Recursively evaluates files inside the folder.
File	C:\Logs\old.log, no	Evaluates that file directly.

Comments and blank lines are ignored. Lines beginning with ; or # are treated as comments.

config/logging.json

Optional logging configuration:

```
{
  "DEBUG": false,
  "COUNT": true,
  "INFO": true,
  "WARN": true,
  "ERROR": true,
  "SUCCESS": true,
  "FATAL": true
}
```

Unknown log levels default to enabled.

Backup Layout

Backups are written under a run-date folder:

```
<backupRoot>/<DDMmmYY>/<folder-name>/<relative-path>/<filename>
```

Example:

```
Source:
C:\Data\Images\2024\Camera\IMG001.jpg

Backup:
D:\backups\30Jan26\Images\2024\Camera\IMG001.jpg
```

The active backup path builder preserves the source folder structure under the dated backup folder. The worker supplies files discovered from the configured source root, and the copy layer creates the destination folders as needed.

Backup Space Validation

When backup is enabled for a path, the worker tracks the size of backup-enabled jobs and checks available space on the backup destination.

The worker performs two checks:

1. Queue-level check: validates the total size of pending backup jobs.
2. Per-file check: validates available destination space immediately before copying a file.

If available backup space is insufficient, the worker cancels the run and does not delete the source file.

Current platform limitation: this validation is implemented for Windows. Linux and macOS currently return a `disk space check not implemented for this platform` error when backup-enabled work reaches this validation.

Logging

Default file logs:

```
logs/maintenance_YYYY-MM-DD.log # all enabled levels
logs/errors_YYYY-MM-DD.log      # ERROR/FATAL-focused output
logs/count_YYYY-MM-DD.log       # summary counts
```

Per-path delete counts are logged after processing completes so totals reflect actual successful deletions.

OS-Specific Platform Abstraction

The platform layer lives under `internal/platform`:

```
internal/platform/
platform.go
current_windows.go
current_linux.go
current_darwin.go
windows/
linux/
macos/
```

The core application depends on the `Platform` interface instead of importing OS-specific packages directly.

Current responsibilities:

- `ShowCritical(title, message string)`
- `DefaultConfigDir(appName string) (string, error)`
- `DefaultLogDir(appName string) (string, error)`
- `EnsureConfig(configDir string, exeDir string) (bool, error)`
- `AvailableBytes(path string) (uint64, error)`

Windows provides the setup wizard and disk-space implementation. Linux and macOS currently perform a safe config existence check only and do not implement backup destination free-space checks yet.

Windows Task Scheduler Setup

Recommended Task Scheduler action fields for the current Windows deployment:

Task Scheduler field	Value
Program/script	<code>powershell.exe</code>
Add arguments	<code>-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -Command "& 'C:\FileCopy\fileMaintenance\fileMaintenance.exe' -days 0 -walkers 1 -max-runtime 55m -cooldown 50ms"</code>
Start in	<code>C:\FileCopy\FileMaintenance</code>

Equivalent PowerShell command, split for readability:

```
$exe = "C:\FileCopy\fileMaintenance\fileMaintenance.exe"
$args = "-days 0 -walkers 1 -max-runtime 55m -cooldown 50ms"

powershell.exe -NoProfile `
  -ExecutionPolicy Bypass `
  -WindowStyle Hidden `
  -Command "& '$exe' $args"
```

Recommended task options:

- Run whether user is logged on or not, if the configured paths and backup destination are available to that account.
- Run as soon as possible after a missed start.
- Stop the task if it runs longer than expected.
- Review `config.ini` because non-zero values in `[settings]` and `[advanced]` override these CLI arguments.

For the current scheduled command:

- `-days 0` makes almost all files older than the current instant eligible.
- `-walkers 1` keeps folder discovery serialized.
- `-max-runtime 55m` caps the run at 55 minutes unless overridden by `config.ini`.
- `-cooldown 50ms` adds a short pause after each processed job unless overridden by `config.ini`.

GitHub Actions Release Build

The GitHub Actions workflow is located at:


```
.github/workflows/build.yml
```

It currently:

- Runs on pull requests to `main`.
- Runs on pushed tags matching `v*`.
- Supports manual `workflow_dispatch` runs.
- Runs `go test ./...` before building each OS/architecture target.
- Builds platform-specific archives:
 - Windows amd64: `.zip`
 - Linux amd64: `.tar.gz`
 - macOS amd64: `.tar.gz`
 - macOS arm64: `.tar.gz`
- Injects version metadata through linker flags:
 - `Version`
 - `Commit`
 - `BuildDate`
- Creates a GitHub release for tag pushes when release assets are available.

Check build metadata locally or from a release binary with:

```
.\fileMaintenance.exe -version
```

Safety Guarantees

- No deletion occurs if backup is enabled and the backup root is inaccessible.
- No deletion occurs if backup copy fails.
- File operations are serialized to reduce network and disk contention.
- Resource controls prevent unbounded walking or job queue growth.
- Critical backup-location failures trigger platform-specific user notification.

Development and Testing

The project currently targets the Go version declared in `go.mod`.

Run tests:

```
go test ./...
```

Build:

```
.\build.ps1 build
```

Run locally:

```
.\build.ps1 run -Days 0
```

Note: the current `build.ps1 smoke` target still checks for legacy `config/folders.txt` and `config/backup.txt` files. The application runtime now uses `config/config.ini`, so update the smoke task before relying on it as the primary local validation path.

License

Internal / private use.